



SYSTEM DESIGN ASSISTANT · DOCUMENT 9

Interaction & **control** contract

The agreed plan-of-record before any application code is written. It fixes how the tool is controlled, how components and projects are modelled, what the live specification must contain, and how an AI agent drives the design — so that we build from a verified contract, not guesses.

• About this document

STATUS

Approved planning contract for the application layer (the engine + content are already built and validated; see doc-8). Everything here is implementable with the existing engine. Terms are defined in the glossary.

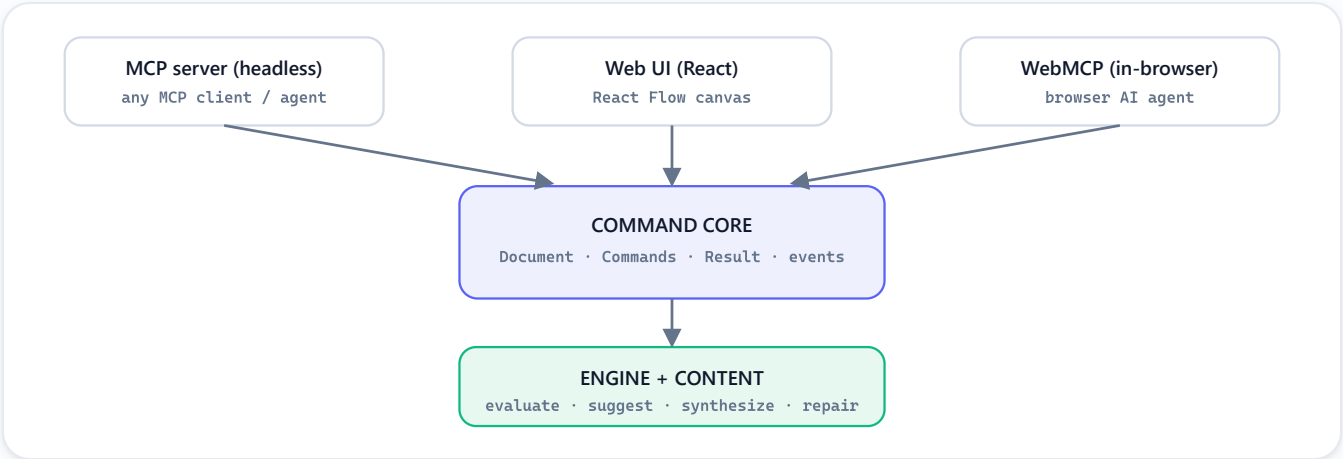
The product is a tool to design software architecture on a canvas and **verify it by computing it** — faster and more beautiful than a plain diagram tool, and uniquely **controllable by AI agents**. This contract turns the decisions we reached into a single reference.

1 The foundational principle — a controllable core, MCP-first

There is exactly one way to change a design: issue a command to the core. Every surface — an AI over MCP, the web UI, an in-browser agent — is just a client of that same command set.

The **command core** is the single source of truth. It holds the project document, applies typed commands (each returning a success-or-errors result), emits change events, and serialises to and from disk. It calls the engine to compute verdicts. It is pure and headless, so it is fully

testable in Node without a browser — and so the **entire flow is driven by MCP first, as the foundation**, with the user interface added afterwards as another client. Because human actions and AI actions are the very same commands, anything a person can do, an agent can do, and vice versa.



One command core; three thin clients above it; the verification engine below. Build bottom-to-top.

2 The command surface

One API, exposed identically to the MCP server, WebMCP, and the UI. Every command returns an explicit result (success with the new state, or a list of errors) — never silent corruption.

GROUP	COMMANDS & QUERIES
Project	<code>new</code> · <code>open / load</code> · <code>save</code> · <code>export</code> · <code>import</code>
Design (instances)	<code>addComponent(type)</code> · <code>remove</code> · <code>connect(portA, portB)</code> · <code>disconnect</code> · <code>setConfig(node, key, value)</code> · <code>setSLO(node, key, band)</code> · <code>move</code>
Component definitions	<code>component.create</code> · <code>read</code> · <code>update</code> · <code>delete</code> · <code>list</code> · <code>import</code> · <code>export</code>
Vocabulary	<code>protocol.add</code> (open) · <code>key.list</code> (governed — algebra is content, not free CRUD)
Engine	<code>evaluate</code> · <code>check</code> · <code>suggest(port)</code> · <code>synthesize(goals)</code> · <code>optimize(objective)</code> · <code>repair</code> · <code>simulate</code>

3 The component model

Two distinct things share the word "component"; we keep them separate.

A **component instance** is a node placed in a design. A **component definition** (a manifest) is the reusable type — and users can author their own. Definitions are pure data, so they are created, edited, imported and exported through commands, and they live in two scopes:

- **Project-scoped** — a custom definition is saved and loaded *with the project*, so the project is self-contained.
- **Catalog-scoped** — a definition shared across projects, in the governed catalogue.

Three rules make this safe: **export bundles dependencies** (a project file carries the definitions it uses, with versions, so it opens anywhere); **every create/update is validated** against the manifest contract (ports, protocols, relations, registry keys) and returns honest errors; and definitions are **namespaced and versioned** (`custom:<org>/<name>@<version>`) to avoid collisions. Note the vocabulary split: **protocols are open** (mere compatibility data a user may add), while **registry keys are governed** because each carries a path-aggregation algebra.

Catalogue & filtering (hundreds of components)

Each definition declares facets — `kind` (compute, data, network, messaging...), `provider` (aws, gcp, azure, oss, generic, `custom:<org>`), and free-form `tags` . The palette offers **faceted, persistent filters**: facets combine as AND across, OR within, plus text search; chosen filters are remembered across sessions. This is content metadata only — the engine is untouched.

4 Interaction — the editor proposes, it does not just accept drops

The canvas (React Flow) is the surface; the intelligence is the engine. This is the moat a plain diagram tool cannot copy.

- **Click-+ on an open port** → `suggest(port)`: the relational layer filters the catalogue to legally-compatible components, then ranks them by pattern affinity and — uniquely — by **SLO impact** (the engine evaluates the design with each candidate and prefers the one that moves you toward your targets).
- **Intent-first** → `synthesize(goals)`: state endpoints and targets; the enumerator generates candidate architectures, the engine verifies each, and ranked, SLO-meeting designs come back.
- **Repair-as-you-go** → `repair`: a failing verdict yields the minimal change that fixes it ("add a read replica"), not just a warning.
- **Pattern templates** and a **constraint-driven palette** (only legally-attachable components are offered) round it out.

Canvas visualisation — animated edges, driven by real metrics

Edge animation encodes computed values across three independent channels, so the eye-candy is also true:

 colour = downstream saturation (the bottleneck glows red)  flow rate = effective throughput

 packet transit = that hop's latency

Because effective throughput along a path equals the bottleneck's throughput, the slow hop sets the tempo of the whole chain — fix it and the entire path speeds up. Unknown values render static, never a faked speed; a small legend keeps it readable.

5 The live design specification

A continuously-updated document beside the canvas, matching what professional architecture documents require — verified, not guessed. Grounded in arc42, the C4 model, the AWS Well-Architected pillars, and ISO/IEC/IEEE 42010.

REQUIRED SECTION (SOURCE)	FILLED BY SDA	MODE
Context & scope (arc42 §3 · C4 L1)	external endpoints of the graph	● computed
Building-block / Container (arc42 §5 · C4 L2)	the placed components	● computed

REQUIRED SECTION (SOURCE)	FILLED BY SDA	MODE
Runtime view (arc42 §6)	request paths + DES dynamics	● computed
Quality requirements / NFRs (arc42 §10)	SLO bands you set → verified live	● provided + verified
Reliability · Performance · Cost (Well-Architected)	availability/SPOF, throughput/latency, cost	● computed
Risks & technical debt (arc42 §11)	the engine's violations & warnings	● computed
Architecture decisions / ADRs (arc42 §9)	recorded by the user; changes logged	● provided
Stakeholders & concerns (42010)	provided; SLOs map to concerns	● provided
Glossary (arc42 §12)	from the registry + protocols	● computed
Security / IAM (Well-Architected)	—	● out of scope

HONESTY CONTRACT

Every value in the spec is tagged **provided**, **computed**, or **unknown** — never a guess. Percentile targets stay unknown until a simulation produces them. The spec is exportable as the official architecture document.

6 AI control — three local-first channels

All three drive the same command core; all are local-first and opt-in, with no data leaving the machine by default.

- **WebMCP (in-browser, core feature)** — the page declares its tools via the emerging browser standard (`navigator.modelContext`); an in-browser AI agent calls them directly, no DOM-scraping.
- **Engine/Document MCP server (headless)** — any MCP client drives the design over a local Node server, operating on a project file or session; no browser needed.

- **Live-tab bridge (optional)** — a local WebSocket lets an external agent drive the running canvas and read on-screen verdicts.

The agent receives **real computed verdicts**, so it runs the verified-design loop — propose, evaluate, repair, re-evaluate — rather than clicking a dumb diagram.

7 Cross-cutting guarantees

- **Undo / redo & history** — free from the command log; shared by human and agent.
- **Single source + serialised commands** — one document; commands applied in order, so concurrent UI + agent edits never diverge.
- **Event stream** — UI and agents subscribe to document/verdict changes; the live spec and animated canvas update themselves.
- **Result on every command** — success-or-errors; the honesty contract at the API level.
- **Versioning & migration** — export records schema/component/plugin versions; import migrates.
- **Consent & permissions** — read vs mutate tools; destructive ops guarded; exposure opt-in; zero egress by default.
- **Command-replay tests** — recorded command scripts become golden tests of whole flows.

8 Build order

- 1 **Command core** (`app/core`) — Document, commands, Result, serialize, engine wiring. Node-tested, including a command-replay golden.
- 2 **MCP server** (`app/mcp`) — headless control of the whole flow over MCP.
- 3 **Web UI** (`app/web`) — React + React Flow, styled A (light) / C (dark): palette + canvas + inspector + live spec + animated edges.
- 4 **WebMCP + adapters** — in-browser tools over the same commands; optional live-tab bridge.

9 Glossary

Command core

The headless heart of the app: holds the project document, applies typed commands, emits events, serialises to disk, and calls the engine. The single source of truth every client uses.

Command vs query

A command changes the document (and returns success-or-errors); a query reads it (verdicts, spec, search). Both are part of the one API.

Component instance vs definition

An instance is a node placed in a design; a definition (manifest) is the reusable type. Users can author definitions.

Manifest

The data file describing a component definition: its ports, configuration, derived formulas and SLO bands.

Facet · faceted filter

An independent dimension of the catalogue (kind, provider, tag). Faceted filtering combines facets (AND across, OR within) plus text, and persists across sessions.

Protocol vs registry key

A protocol is the language a port speaks (open data, user-addable). A registry key is a property the engine combines along a path (governed, because it carries an aggregation algebra).

SLO Service Level Objective

A target the design must meet (e.g. p99 latency < 200 ms). Set by the user, verified live by the engine.

Verdict · cause-chain · remediation

The engine's answer for one target (pass/warn/fail/unknown), the trace to the component responsible, and a ranked suggested fix.

Saturation

How close a component is to its capacity ($\text{flow} \div \text{capacity}$); at or above 1 it is the bottleneck. Drives the red edge colour.

MCP Model Context Protocol

A standard by which an AI agent calls a tool's structured functions. SDA exposes its commands as MCP tools.

WebMCP

An emerging browser standard (`navigator.modelContext`) by which a web page declares callable tools to an in-browser AI agent.

Synthesize · optimize · repair

The "reverse" engine modes: generate & rank candidate designs; find the cheapest meeting the targets; find the smallest change that fixes a failing one.

Command-replay test

A recorded sequence of commands re-run as an automated test of a whole user flow.